
Phân công code vấn đáp đồ án

X509Certificate

Phần việc chi tiết của thành viên

Nguyễn Lê Anh Kiệt

*Vòng đời chứng chỉ, Thu hồi
và Hạ tầng CRL & OCSP*

Tài liệu này mô tả chi tiết phần code, lý thuyết X.509 cần nắm, các test case và cách tích hợp cho phần việc **vòng đời chứng chỉ (lifecycle)**, **thu hồi (revocation)** và **hạ tầng phân phối trạng thái CRL & OCSP**, gắn trực tiếp với mã nguồn thực tế của đồ án trong các thư mục `src/core/` (`crl.py`), `src/services/` (`cert_lifecycle.py`, `revocation_workflow.py`, `crl_publish.py`, `infra_manager.py`), `src/infra/` (`crl_server.py`, `ocsp_server.py`) và các màn hình UI tương ứng.

Môn: Mã hoá và Ứng dụng (MHUD)

Học phần đồ án X.509 Certificate

Ngày biên soạn: 14/06/2026

Contents

1	Mục tiêu và phạm vi phần việc	2
1.1	Ảnh xạ khái niệm → file code → vai trò	2
2	Kiến trúc tổng thể và vị trí của thành viên trong hệ thống	2
2.1	Luồng thu hồi end-to-end	2
2.2	Bảng vị trí module trong toàn hệ thống	4
3	Phần code cần làm (chi tiết)	4
3.1	Vòng đời cert: suy ra trạng thái động	4
3.2	Revoke và Renew một chứng chỉ	5
3.3	Workflow thu hồi hai phía	6
3.4	Lỗi CRL và OCSP DB (core/crl.py)	7
3.5	Snapshot và publish CRL (crl_publish.py)	8
3.6	Hạ tầng máy chủ: CRL server	9
3.7	Hạ tầng máy chủ: OCSP responder	10
3.8	Quản lý vòng đời 4 server: infra_manager	10
3.9	Giao diện người dùng (UI)	11
4	Cần nắm để vấn đáp	12
5	Test case và demo	13
5.1	Bảng test case chính	13
5.2	Đoạn unit test gợi ý	13
5.3	Kịch bản demo cho hội đồng	15
6	Tích hợp vào chương trình chung	15
7	Bổ sung sau rà soát: các hàm/feature dễ bị hỏi	16
8	Checklist chuẩn bị vấn đáp	16

1 Mục tiêu và phạm vi phân việc

Theo bảng phân công chung, thành viên **Nguyễn Lê Anh Kiệt** phụ trách cụm chức năng **vòng đời chứng chỉ, thu hồi và hạ tầng CRL & OCSP**. Đây là phần *nằm ở khâu hậu cấp phát*: sau khi một chứng chỉ đã được Root CA ký và phát hành cho khách hàng, nó vẫn cần được quản lý suốt vòng đời còn lại — gia hạn (renew), thu hồi (revoke) khi khoá bị lộ hay không còn dùng nữa, và quan trọng nhất là *công bố trạng thái thu hồi ra ngoài* để bất kỳ bên nào cũng kiểm tra được một cert còn hợp lệ hay không.

Cụm này trả lời câu hỏi cốt lõi của PKI: “*Làm sao biết một chứng chỉ tuy chưa hết hạn nhưng đã bị thu hồi?*”. Câu trả lời gồm hai cơ chế bù nhau: **CRL** (Certificate Revocation List — danh sách thu hồi do CA ký, phát hành định kỳ) và **OCSP** (Online Certificate Status Protocol — tra cứu trạng thái từng serial theo thời gian thực). Phần việc của Kiệt hiện thực trọn vẹn cả hai từ tầng nghiệp vụ (services) tới tầng máy chủ HTTP (infra) và giao diện người dùng (UI) cho cả Admin lẫn Customer.

1.1 Ánh xạ khái niệm → file code → vai trò

Bảng dưới đây liên kết từng khái niệm lý thuyết với file mã nguồn thực tế và vai trò của nó trong hệ thống. Đây là bản đồ tổng quát để định vị nhanh khi vấn đáp.

Phạm vi đáp ứng yêu cầu đề bài. Cụm này hiện thực các hạng mục A.8 (Admin quản lý cert: revoke/renew), A.9 (Admin duyệt yêu cầu thu hồi), A.10 (cập nhật CRL), B.6 (Customer xem/tải cert), B.7 (Customer xin thu hồi), B.8 (Customer tra CRL công khai) và một phần B.9 (hạ tầng để verify dựa vào CRL/OCSP).

2 Kiến trúc tổng thể và vị trí của thành viên trong hệ thống

Hệ thống chia thành 3 tầng rõ rệt: tầng **lõi mật mã** (core/) chỉ làm việc với đối tượng cryptography; tầng **ng nghiệp vụ** (services/) chứa logic quy trình + truy vấn DB SQLite; tầng **hạ tầng + UI** (infra/, ui/) cung cấp HTTP server và màn hình tương tác. Cụm của Kiệt trải khắp cả 3 tầng theo một luồng dữ liệu thống nhất.

2.1 Luồng thu hồi end-to-end

Sơ đồ dưới đây mô tả luồng từ lúc khách hàng xin thu hồi cho tới lúc CRL/OCSP phản ánh trạng thái mới ra bên ngoài.

Customer chọn cert active + lý do
(`revoke_request_view` →
`submit_revoke_request`)

Khái niệm	File code	Vai trò
Vòng đời cert (status động)	<code>services/cert_lifecycle.py</code>	Suy ra <code>active/expired/revoked</code> , liệt kê cert, revoke, renew.
Workflow thu hồi 2 phía	<code>services/revocation_workflow.py</code>	Customer gửi yêu cầu thu hồi; Admin duyệt (approve/reject) atomic.
Lỗi CRL & OCSP DB	<code>core/crl.py</code>	Build/ký CRL bằng cryptography; đọc-ghi OCSP DB (JSON realtime).
Snapshot & publish CRL	<code>services/crl_publish.py</code>	Chụp serial revoked từ DB, ký CRL bằng Root CA, ghi file + sync OCSP.
Máy chủ phân phối CRL	<code>infra/crl_server.py</code>	HTTP GET <code>/crl.pem</code> để client tải CRL về đối chiếu.
OCSP responder	<code>infra/ocsp_server.py</code>	HTTP GET <code>/ocsp?serial=</code> trả GOOD/REVOKED; flag <code>enabled</code> giả lập down.
Quản lý vòng đời 4 server	<code>services/infra_manager.py</code>	Singleton start/stop idempotent cặp Prod (8889/8888) + Lab (9889/9888).
UI quản lý cert (Admin)	<code>ui/admin/cert_mgmt_view.py</code>	Bảng cert, dialog Revoke + Renew.
UI duyệt thu hồi (Admin)	<code>ui/admin/revoke_queue_view.py</code>	Queue request, Approve (auto publish CRL) / Reject.
UI cập nhật CRL (Admin)	<code>ui/admin/crl_publish_view.py</code>	Nút “Publish CRL Now”, xem diff DB vs CRL hiện hành.
UI cert của tôi (Customer)	<code>ui/customer/my_certs_view.py</code>	Customer xem + tải cert đã cấp.
UI xin thu hồi (Customer)	<code>ui/customer/revoke_request_view.py</code>	Customer chọn cert active + lý do → gửi yêu cầu.
UI tra CRL (Customer)	<code>ui/customer/view_crl_view.py</code>	Tra cứu danh sách thu hồi công khai theo serial/CN.

↓ tạo bản ghi `revocation_requests` status=`pending`

Admin mở queue, bấm Approve
(`revoke_queue_view` → `approve_revocation`)

↓ trong 1 transaction: set request=`approved` + UPDATE `issued_certs.revoked_at`

`sync_ocsp_db` ghi `ocsp_db.json` (realtime)
+ `publish_crl` ký CRL bằng Root CA →
`crl.pem`

↓

Hạ tầng phục vụ trạng thái mới:
`crl_server` GET `/crl.pem` | `ocsp_server`
GET `/ocsp?serial=`

Điểm mấu chốt cần nhớ khi vấn đáp: có **hai nguồn sự thật tách biệt**. `ocsp_db.json` luôn được cập nhật *ngay* mỗi khi có revoke (qua `sync_ocsp_db`), nên OCSP responder luôn fresh. Còn file `crl.pem` chỉ thay đổi khi có hành động *Publish CRL* (thủ công hoặc tự động

sau approve). Sự lệch pha này chính là mô phỏng đúng thực tế: CRL có “cửa sổ trễ” giữa hai lần phát hành, còn OCSP gần như tức thời.

2.2 Bảng vị trí module trong toàn hệ thống

Tầng	Module của Kiệt	Phụ thuộc / được gọi bởi
Core	core/crl.py	dùng cryptography; được crl_publish và ocsd_server gọi.
Services	cert_lifecycle, revocation_workflow, crl_publish	gọi core/crl, ca_admin (load Root CA), db.connection.
Services	infra_manager	gọi infra/crl_server + infra/ocsp_server; được ui/app.py gọi lúc boot.
Infra	crl_server, ocsd_server	dùng http.server; ocsd_server đọc OCSP DB qua core.crl.
UI Admin	3 view	gọi services lifecycle/workflow/publish + services.audit.
UI Customer	3 view	gọi services hoặc Admin API (chế độ remote).

3 Phần code cần làm (chi tiết)

Phần này đi qua từng nhóm hàm quan trọng, kèm code thật trích từ mã nguồn và giải thích từng ý. Mỗi sinh viên phụ trách cần đọc kỹ để trả lời được câu hỏi “hàm này làm gì, vì sao viết như vậy”.

3.1 Vòng đời cert: suy ra trạng thái động

Điểm thiết kế cốt lõi của cert_lifecycle.py là status không lưu trong DB mà được tính lại mỗi lần đọc dựa trên hai cột thô: revoked_at và not_valid_after.

```

1 def _compute_status(row, now: datetime) -> str:
2     """Suy ra status hiện tại của cert."""
3     if row["revoked_at"]:
4         return "revoked"
5     na = _parse_iso(row["not_valid_after"])
6     if na is None:
7         return "active"
8     if na.tzinfo is None:                # not_valid_after lưu tz-aware;
9         na = na.replace(tzinfo=timezone.utc)  # nếu không, gán UTC.
10    if na < now:
11        return "expired"
12    return "active"

```

Giải thích từng ý:

- Thứ tự kiểm tra có chủ đích: **revoked** được ưu tiên cao nhất — một cert đã thu hồi thì dù còn hạn vẫn báo **revoked**.
- Nếu chưa revoke, so sánh **not_valid_after** với **now**: quá hạn → **expired**, ngược lại → **active**.
- Xử lý timezone tỉ mỉ: nếu chuỗi ISO không kèm offset thì gán UTC để tránh lỗi so sánh “naive vs aware datetime”.

Vì sao tính động thay vì lưu cột status? Vì **expired** thay đổi theo thời gian mà *không* có sự kiện nào ghi vào DB — một cert tự “hết hạn” khi đồng hồ vượt mốc. Nếu lưu cứng, ta phải có job quét định kỳ cập nhật, dễ sai lệch. Tính động đảm bảo status luôn đúng tại thời điểm truy vấn.

Hàm `_list_certs` áp dụng filter status *post-hoc* (sau khi đã tính status) chứ không đưa vào câu `WHERE SQL`, đúng vì status là trường suy ra:

```

1 now = datetime.now(timezone.utc)
2 with conn_scope(db_path) as conn:
3     rows = conn.execute(sql, params).fetchall()
4     out = []
5     for r in rows:
6         d = _row_to_dict(r, now)          # gán d["status"]
7         if status in (None, "all") or d["status"] == status:
8             out.append(d)
9     return out

```

3.2 Revoke và Renew một chứng chỉ

`revoke_cert` đánh dấu thu hồi: validate lý do, chặn revoke trùng (giữ duy nhất một sự kiện thu hồi), ghi `revoked_at` trong transaction rồi *sync ngay* OCSP DB.

```

1 now = datetime.now(timezone.utc).isoformat()
2 with transaction(db_path) as conn:
3     row = conn.execute(
4         "SELECT revoked_at FROM issued_certs WHERE id = ?", (cert_id,),
5     ).fetchone()
6     if row is None:
7         raise CertLifecycleError("Không tìm thấy cert.")
8     if row["revoked_at"]:
9         raise CertLifecycleError(f"Cert đã bị revoked lúc {row['revoked_at']}.")
10    conn.execute(
11        "UPDATE issued_certs SET revoked_at = ?, revocation_reason = ? "
12        "WHERE id = ?", (now, reason, cert_id),
13    )
14    sync_ocsp_db(db_path, ocsp_db_path)    # OCSP fresh ngay, CRL publish sau

```

```
15 return get_cert_detail(cert_id, db_path)
```

`renew_cert` *phát hành cert mới* giữ nguyên subject + public key của cert cũ, hiệu lực mới, không tự revoke cert cũ. Đây là khác biệt then chốt với “reissue qua CSR mới”:

```
1 old = get_cert_detail(cert_id, db_path)
2 if old["revoked_at"]:
3     raise CertLifecycleError("Cert đã bị thu hồi - không thể renew.")
4 ca_cert, ca_key = load_active_root_ca_with_key(db_path)
5 old_cert_obj = x509.load_pem_x509_certificate(bytes(old["cert_pem"]))
6 new_cert, new_serial = reissue_cert_for_renewal(
7     old_cert_obj, ca_cert, ca_key,
8     validity_days=validity_days, ocsf_url=ocsf_url, crl_url=crl_url,
9 )
10 # INSERT cert mới với renewed_from_id = cert_id (truy vết chuỗi gia hạn)
```

Cột `renewed_from_id` lưu liên kết cert mới → cert cũ, cho phép truy vết toàn bộ “chuỗi gia hạn” của một domain. UI hiển thị cột “Renew từ” chính là dựa vào trường này.

3.3 Workflow thu hồi hai phía

`revocation_workflow.py` tách quyền: **customer chỉ được xin**, còn **admin mới được quyết**. Phía customer, `submit_revoke_request` chứa một *BOLA guard* (Broken Object Level Authorization) quan trọng:

```
1 cert = conn.execute(
2     "SELECT id, owner_id, revoked_at, common_name "
3     "FROM issued_certs WHERE id = ?", (cert_id,),
4 ).fetchone()
5 if cert is None or cert["owner_id"] != requester_id:
6     raise RevocationWorkflowError(
7         f"Cert id={cert_id} không tồn tại hoặc không thuộc về bạn.")
8 if cert["revoked_at"]:
9     raise RevocationWorkflowError("Cert đã bị thu hồi rồi, không cần gửi yêu cầu.")
10 # Chặn trùng: 1 cert chỉ có tối đa 1 request pending
11 dup = conn.execute(
12     "SELECT id FROM revocation_requests "
13     "WHERE issued_cert_id = ? AND status = 'pending'", (cert_id,),
14 ).fetchone()
15 if dup:
16     raise RevocationWorkflowError(f"Đã có yêu cầu pending (request #{dup['id']}).")
```

Giải thích: kiểm tra `owner_id != requester_id` ngăn người dùng A xin thu hồi cert của người dùng B (lỗ hổng phân quyền cấp đối tượng cổ điển). Việc chặn request trùng giữ queue admin sạch.

Phía admin, `approve_revocation` là điểm *atomic* chống race khi hai admin cùng duyệt một request. Toàn bộ thay đổi DB nằm trong một transaction:

```

1 with transaction(db_path) as conn:
2     req = conn.execute("SELECT id, issued_cert_id, reason, status "
3         "FROM revocation_requests WHERE id = ?", (req_id,)).fetchone()
4     if req is None:
5         raise RevocationWorkflowError("Không tìm thấy request.")
6     if req["status"] != "pending":
7         raise RevocationWorkflowError(
8             f"Request đang ở status '{req['status']}', không approve được.")
9     cert = conn.execute("SELECT id, revoked_at FROM issued_certs WHERE id = ?",
10         (req["issued_cert_id"],)).fetchone()
11     if cert["revoked_at"] is None:          # chỉ ghi nếu chưa revoke
12         conn.execute("UPDATE issued_certs SET revoked_at = ?, "
13             "revocation_reason = ? WHERE id = ?", (now, req["reason"], cert["id"]))
14         cert_was_revoked = True
15     else:
16         cert_was_revoked = False          # admin đã revoke trực tiếp trước đó
17     conn.execute("UPDATE revocation_requests SET status = 'approved', "
18         "reviewed_at = ?, reviewed_by = ? WHERE id = ?", (now, admin_id, req_id))

```

Sau khi commit transaction, hàm gọi `sync_ocsp_db` (OCSP fresh ngay) và nếu caller truyền `crl_path` thì gọi luôn `publish_crl` để CRL tự cập nhật:

```

1 sync_ocsp_db(db_path, ocsp_db_path)
2 crl_result = crl_error = None
3 if crl_path:
4     try:
5         crl_result = publish_crl(admin_id=admin_id, db_path=db_path,
6             crl_path=crl_path, ocsp_db_path=ocsp_db_path)
7     except CRLPublishError as e:
8         crl_error = str(e)
9 return {"id": req_id, "cert_was_revoked": cert_was_revoked,
10     "crl_result": crl_result, "crl_error": crl_error, ...}

```

Điểm tinh tế về race re-check. Nếu admin đã revoke cert trực tiếp ở màn A.8 trước đó, khi approve request hệ thống *vấn* đổi request sang **approved** (để khép audit) nhưng *không ghi* đề `revoked_at` cũ — giữ đúng thời điểm thu hồi đầu tiên. Đây là re-check trạng thái cert ngay trong transaction.

3.4 Lỗi CRL và OCSP DB (`core/crl.py`)

`build_crl` dựng đối tượng CRL bằng builder của thư viện `cryptography` và ký bằng khoá riêng của issuer (Root CA):

```

1 def build_crl(issuer_cert, issuer_key, revoked_serials, validity_days=7):

```



```

2     now = datetime.now(timezone.utc)
3     builder = (x509.CertificateRevocationListBuilder()
4                 .issuer_name(issuer_cert.subject)
5                 .last_update(now)
6                 .next_update(now + timedelta(days=validity_days)))
7     for serial in revoked_serials:
8         revoked = (x509.RevokedCertificateBuilder()
9                     .serial_number(int(serial)).revocation_date(now).build())
10        builder = builder.add_revoked_certificate(revoked)
11    return builder.sign(private_key=issuer_key, algorithm=hashes.SHA256())

```

OCSP DB là một file JSON đơn giản chứa danh sách serial bị thu hồi. Hai hàm đọc-ghi và một hàm dùng riêng để *demo khoảng cách CRL vs OCSP*:

```

1  def load_revoked_list(path: str) -> set:
2      """Đọc danh sách serial bị revoke từ JSON. Trả về set[int]."""
3      if not os.path.exists(path):
4          return set()
5      with open(path) as f:
6          data = json.load(f)
7      return set(int(s) for s in data)
8
9  def revoke_serial_ocsp_only(serial: int, ocsp_db_path: str):
10     """Thêm serial vào OCSP DB mà KHÔNG cập nhật CRL."""
11     revoked = load_revoked_list(ocsp_db_path)
12     revoked.add(serial)
13     save_revoked_list(list(revoked), ocsp_db_path)

```

`build_and_publish_crl` là tiện ích snapshot toàn bộ OCSP DB rồi ký CRL — tương đương bấm nút “Publish CRL Now”.

3.5 Snapshot và publish CRL (`crl_publish.py`)

`snapshot_revoked_serials` đọc tất cả cert có `revoked_at` khác NULL, parse `serial_hex` (hex) sang int. Đáng chú ý: *không* loại trừ cert đã hết hạn — đúng chuẩn CRL.

```

1  def snapshot_revoked_serials(db_path: str) -> "list[int]":
2      with conn_scope(db_path) as conn:
3          rows = conn.execute(
4              "SELECT serial_hex FROM issued_certs "
5              "WHERE revoked_at IS NOT NULL ORDER BY revoked_at ASC").fetchall()
6      out = []
7      for r in rows:
8          try:
9              out.append(int(r["serial_hex"], 16))
10         except (ValueError, TypeError):
11             print(f"[crl_publish] WARN: serial_hex không parse được: "
12                   f"{r['serial_hex']!r}", file=sys.stderr)

```

```
13     return out
```

`publish_crl` là trái tim của A.10: load Root CA active, snapshot serial, build + ký CRL, ghi file, đồng bộ OCSP, trả metadata cho UI.

```
1  def publish_crl(admin_id, db_path, crl_path=DEFAULT_CRL_PATH,
2      oosp_db_path=DEFAULT_OCSP_DB_PATH, validity_days=7):
3      try:
4          ca_cert, ca_key = load_active_root_ca_with_key(db_path)
5      except CAError as e:
6          raise CRLPublishError(f"Không publish được CRL: {e} Tạo Root CA trước.")
7      serials = snapshot_revoked_serials(db_path)
8      crl = build_crl(ca_cert, ca_key, serials, validity_days=validity_days)
9      os.makedirs(os.path.dirname(crl_path) or ".", exist_ok=True)
10     save_crl(crl, crl_path)
11     if oosp_db_path:
12         sync_oosp_db(db_path, oosp_db_path)
13     return {"revoked_count": len(serials), "crl_path": os.path.abspath(crl_path),
14         "this_update": ..., "next_update": ...,
15         "issuer": ca_cert.subject.rfc4514_string()}
```

`sync_oosp_db` tách riêng vì *không cần ký* — chỉ ghi JSON, nên revoke flow gọi được tức thì mà không phụ thuộc Root CA. `list_crl_entries` và `get_published_crl_info` parse lại file CRL để UI hiển thị; bản trước còn JOIN `issued_certs` làm giàu thêm CN + owner cho dễ đọc.

3.6 Hạ tầng máy chủ: CRL server

`crl_server.py` dùng factory pattern: mỗi lần gọi `start_crl_server` tạo một Handler class riêng capture `crl_path` qua closure, cho phép chạy nhiều instance (prod + lab) trên port khác nhau.

```
1  class CRLHandler(BaseHTTPRequestHandler):
2      def do_GET(self):
3          if self.path not in ("/crl.pem", "/crl"):
4              self.send_response(404); self.end_headers(); return
5          if not os.path.exists(crl_path):
6              self.send_response(404); self.end_headers()
7              self.wfile.write(b"CRL not found"); return
8          with open(crl_path, "rb") as f:
9              data = f.read()
10         self.send_response(200)
11         self.send_header("Content-Type", "application/x-pem-file")
12         self.send_header("Content-Length", str(len(data)))
13         self.end_headers(); self.wfile.write(data)
```

Server chạy trên `ThreadingHTTPServer` với `daemon_threads = True`: mỗi request một thread

con, một client chậm không treo các client khác, và thread tự thoát khi process chính kết thúc.

3.7 Hạ tầng máy chủ: OCSP responder

`ocsp_server.py` nhận GET `/ocsp?serial=<int>` và trả JSON GOOD/REVOKED. Flag `enabled` bọc trong một dict mutable để UI *toggle runtime* giả lập OCSP down (trả 503) mà không cần restart server.

```

1 class OCSPHandler(BaseHTTPRequestHandler):
2     def do_GET(self):
3         if not state.get("enabled", True):           # giả lập down
4             body = b'{"error": "OCSP responder temporarily unavailable"}'
5             self.send_response(503); ...; self.wfile.write(body); return
6         parsed = urlparse(self.path)
7         if parsed.path != "/ocsp":
8             self.send_response(404); self.end_headers(); return
9         serial_raw = parse_qs(parsed.query).get("serial", [None])[0]
10        if serial_raw is None:
11            self._json(400, {"error": "missing 'serial' parameter"}); return
12        serial_int = int(serial_raw)
13        revoked = load_revoked_list(revoked_list_path) # đọc JSON realtime
14        status = "REVOKED" if serial_int in revoked else "GOOD"
15        self._json(200, {"serial": str(serial_int), "status": status})

```

Vì sao OCSP đọc file mỗi request? Để luôn fresh. Mỗi khi revoke gọi `sync_ocsp_db` ghi lại JSON, lần tra OCSP kế tiếp đã thấy ngay. Đây chính là lý do OCSP “realtime” còn CRL thì trễ.

3.8 Quản lý vòng đời 4 server: `infra_manager`

`infra_manager.py` là singleton quản hai cặp server: **Prod** (8889/8888, auto-start lúc boot, phục vụ cert thật từ DB) và **Lab** (9889/9888, start thủ công khi mở Verification Lab, phục vụ cert demo). Các thao tác start đều *idempotent* — gọi nhiều lần chỉ start một lần:

```

1 def start_prod_servers(self):
2     os.makedirs(CERTS_DIR, exist_ok=True)
3     if self._prod_crl is None:           # idempotent guard
4         try:
5             self._prod_crl = start_crl_server(host="localhost",
6                 port=PROD_CRL_PORT, crl_path=PROD_CRL_PATH)
7         except OSError as e:
8             _log.warning("Prod CRL server không start được (port %d?): %s",
9                 PROD_CRL_PORT, e)
10    if self._prod_ocsp is None:
11        self._prod_ocsp, self._prod_ocsp_state = start_ocsp_server(

```

```
12         host="localhost", port=PROD_OCSP_PORT, revoked_list_path=PROD_OCSP_DB)
```

Toggle OCSP enabled dựa vào việc `start_ocsp_server` trả về cả *state dict mutable*; manager giữ reference này và sửa trực tiếp field `enabled`:

```
1 def set_lab_ocsp_enabled(self, enabled: bool):
2     if self._lab_ocsp_state is not None:
3         self._lab_ocsp_state["enabled"] = enabled    # handler thấy ngay
```

Singleton lấy qua `get_infra()` (lazy init), đảm bảo cả app chỉ có một instance quản lý vòng đời 4 server suốt process.

3.9 Giao diện người dùng (UI)

Admin — `cert_mgmt_view`: bảng cert tô màu theo status (active xanh, expired xám, revoked đỏ), nút Revoke mở `RevokeCertDialog` (lý do bắt buộc) gọi `revoke_cert`, nút Renew mở `RenewCertDialog` gọi `renew_cert`; mọi thao tác đều ghi `write_audit`.

Admin — `revoke_queue_view`: queue request (mặc định filter `pending`). Khi Approve, gọi `approve_revocation` với cả `crl_path` nên CRL *tự publish*; UI ghi audit `REVOKE_APPROVED`, `CERT_REVOKED` và `CRL_PUBLISHED`, và thông báo số serial trong CRL.

```
1 result = approve_revocation(req_id, self.app.session["id"], self.app.db_path,
2                             ocsp_db_path=DEFAULT_OCSP_DB_PATH, crl_path=DEFAULT_CRL_PATH)
3 ...
4 crl_result = result.get("crl_result")
5 if crl_result:
6     write_audit(..., Action.CRL_PUBLISHED, ...,
7                 details={"revoked_count": crl_result["revoked_count"],
8                           "source": "auto_after_revoke_approve"})
```

Admin — `crl_publish_view`: hiển thị metadata CRL hiện hành, snapshot DB (số serial sẽ vào CRL), *diff* cảnh báo nếu DB lệch CRL, và nút “Publish CRL Now” gọi `publish_crl`.

Customer — `my_certs_view`: xem + tải cert của mình (`list_certs_for_owner`, có chế độ remote qua Admin API). `revoke_request_view`: chọn cert active + lý do, gọi `submit_revoke_request`. `view_crl_view`: tra CRL công khai theo serial/CN qua `list_crl_entries` + `get_published_crl_info`.

4 Cần nắm để vấn đáp

Hỏi: Trạng thái cert được lưu cứng trong DB hay suy ra động? Vì sao?

Đáp: Suy ra động trong `_compute_status`. DB chỉ lưu hai cột thô `revoked_at` và `not_valid_after`; status `active/ expired/revoked` được tính lại mỗi lần đọc. Lý do: `expired` phụ thuộc thời gian hiện tại, không có “sự kiện hết hạn” để ghi DB — tính động luôn đúng, tránh dữ liệu lỗi thời.

Hỏi: CRL và OCSP khác nhau ra sao và bù nhau thế nào?

Đáp: CRL là *danh sách* toàn bộ serial bị thu hồi do CA ký, phát hành định kỳ (`this_update` → `next_update`); client tải về một lần rồi tra offline — ưu điểm là không cần online liên tục, nhược điểm là có độ trễ tới lần publish kế tiếp và file phình to. OCSP tra *từng serial* theo thời gian thực, trả GOOD/REVOKED ngay — ưu điểm tươi mới, nhược điểm là phụ thuộc responder online. Trong đồ án, OCSP DB cập nhật tức thì mỗi lần revoke (`sync_ocsp_db`), còn CRL chỉ đổi khi Publish — minh hoạ đúng sự bù trừ giữa hai cơ chế.

Hỏi: Vì sao CRL phải được ký bằng Root CA?

Đáp: CRL là tuyên bố “những cert này không còn tin cậy”. Nếu không ký, kẻ tấn công có thể giả mạo CRL rỗng để giấu việc một cert đã bị thu hồi, hoặc ngược lại đưa cert hợp lệ vào CRL để từ chối dịch vụ. Chữ ký của Root CA (`builder.sign(issuer_key, SHA256)`) cho phép client xác minh CRL thật sự do CA phát hành và chưa bị sửa. OCSP DB JSON *không ký* chỉ vì nó là kênh nội bộ phục vụ responder, không phát tán trực tiếp như CRL.

Hỏi: Mô tả luồng thu hồi hai phía (customer xin, admin duyệt).

Đáp: Customer gọi `submit_revoke_request` (có BOLA guard kiểm tra sở hữu, chặn cert đã revoked, chặn request pending trùng), tạo bản ghi `pending`. Admin mở queue, Approve gọi `approve_revocation`: trong một transaction set `request=approved` và UPDATE `revoked_at`, sau commit thì sync OCSP + publish CRL. Nếu cert đã revoke trước đó thì giữ nguyên thời điểm cũ. Admin cũng có thể Reject (lý do bắt buộc, append vào reason để giữ history).

Hỏi: `renew_cert` khác “reissue qua CSR mới” ở điểm nào?

Đáp: Renew giữ nguyên *subject + public key* của cert cũ, chỉ đổi serial + hiệu lực mới, ghi `renewed_from_id` để truy vết — customer không phải gửi CSR lại. Reissue qua CSR là cấp một cert hoàn toàn mới (có thể khoá mới). Renew từ chối nếu cert cũ đã thu hồi, và *không* tự revoke cert cũ (admin tự quyết).

Hỏi: Vòng đời các server hạ tầng được quản lý ra sao?

Đáp: `InfraManager` là singleton (lấy qua `get_infra()`). Prod (8889/8888) auto-start lúc boot và chạy suốt đời app; Lab (9889/9888) start thủ công khi mở Verification Lab, stop khi đóng. Start idempotent nhờ guard if `self._prod_crl is None`. `stop_all` gọi shutdown + `server_close` cả 4. OSCP có thể bật/tắt runtime qua state dict mutable.

Hỏi: Vì sao tách `ocsp_db.json` và `crl.pem` thành hai nguồn?

Đáp: Để mô phỏng đúng thực tế PKI. `ocsp_db.json` cập nhật tức thì mỗi revoke (không cần ký) nên OSCP luôn fresh; `crl.pem` chỉ đổi khi Publish (cần ký Root CA) nên có cửa sổ trễ. Màn `crl_publish_view` hiển thị diff giữa số serial trong DB và trong CRL để nhắc admin publish khi lệch.

Lưu ý dễ bị hỏi vặn. `serial_hex` trong DB là chuỗi hex, phải `int(serial_hex, 16)` mới ra số để đưa vào CRL/OCSP. OSCP responder so sánh `serial_int` in `revoked` với `revoked` là `set[int]` — nếu lưu sai kiểu (str vs int) sẽ luôn trả GOOD nhầm. Hãy giải thích được điểm này.

5 Test case và demo

5.1 Bảng test case chính

5.2 Đoạn unit test gợi ý

Test vòng đời + thu hồi (gợi ý cho `test_m7_cert_lifecycle.py` và `test_m8_revocation_crl.py`):

```

1 def test_revoke_then_status_revoked(tmp_db):
2     cert = issue_sample_cert(tmp_db)
3     revoke_cert(cert["id"], admin_id=1, reason="key lộ", db_path=tmp_db)
4     detail = get_cert_detail(cert["id"], tmp_db)
5     assert detail["status"] == "revoked"
6     assert detail["revoked_at"] is not None
7
8 def test_submit_bola_guard(tmp_db):
9     cert = issue_sample_cert(tmp_db, owner_id=10)
10    with pytest.raises(RevocationWorkflowError):
11        submit_revoke_request(cert["id"], requester_id=99,
12                               reason="x", db_path=tmp_db) # không phải chủ
13
14 def test_publish_crl_signs_with_root_ca(tmp_db):
15     cert = issue_sample_cert(tmp_db)
16     revoke_cert(cert["id"], 1, "r", tmp_db)
17     res = publish_crl(admin_id=1, db_path=tmp_db,
```

Test	Kịch bản	Kỳ vọng
compute_status active	cert chưa revoke, còn hạn	status = active
compute_status expired	not_valid_after < now	status = expired
compute_status revoked	revoked_at != NULL	status = revoked (ưu tiên)
revoke_cert twice	revoke 1 cert hai lần	lần 2 raise CertLifecycleError
renew_cert	renew cert active	cert mới, renewed_from_id đúng
renew revoked	renew cert đã revoke	raise lỗi
submit BOLA	customer xin revoke cert người khác	raise (không thuộc về bạn)
submit duplicate	xin revoke khi đã có pending	raise (đã có request)
approve atomic	approve request pending	request=approved + cert revoked
approve twice	approve request đã approved	raise (status không pending)
publish_crl	có 2 cert revoked	CRL ký, revoked_count=2
OCSP GOOD/REVOKED	tra serial chưa/dã revoke	trả GOOD / REVOKED
OCSP disabled	set enabled=False rồi tra	HTTP 503
infra idempotent	gọi start_prod 2 lần	chỉ 1 server, không lỗi

```

18         crl_path=tmp_path/"crl.pem")
19     assert res["revoked_count"] == 1
20     crl = load_crl(tmp_path/"crl.pem")
21     assert crl.is_signature_valid(root_ca_public_key)

```

Test hạ tầng (gợi ý cho test_infra_manager.py):

```

1  def test_start_prod_idempotent():
2      mgr = InfraManager()
3      mgr.start_prod_servers()
4      first = mgr._prod_crl
5      mgr.start_prod_servers()          # gọi lần 2
6      assert mgr._prod_crl is first    # không tạo server mới
7      mgr.stop_all()
8
9  def test_ocsp_toggle_returns_503(ocsp_running):
10     server, state = ocsp_running
11     state["enabled"] = False
12     resp = http_get("http://localhost:8888/ocsp?serial=1")

```

13

```
assert resp.status == 503
```

5.3 Kịch bản demo cho hội đồng

1. **Revoke + publish:** Admin mở Quản lý chứng nhận, chọn một cert active, bấm Revoke (nhập lý do). Mở Cập nhật CRL: thấy diff “DB có 1 revoked, CRL có 0”. Bấm *Publish CRL Now* → CRL ký lại, `revoked_count=1`.
2. **Tra CRL/OCSP:** Customer mở Tra cứu CRL thấy serial vừa revoke. Gọi `GET /ocsp?serial=<serial>` (trình duyệt hoặc curl) trả `REVOKED`; tra một serial active trả `GOOD`.
3. **Luồng hai phía:** Customer mở Yêu cầu thu hồi, chọn cert active + lý do, Gửi. Admin mở Duyệt yêu cầu thu hồi, Approve → cert revoked + CRL tự publish (kiểm tra thông báo số serial).
4. **Giả lập OCSP down:** Trong Verification Lab, tắt OCSP (toggle `enabled=False`); tra OCSP trả HTTP 503 — minh họa vì sao cần CRL làm phương án dự phòng khi responder gặp sự cố.

Lệnh chạy test. Từ thư mục gốc dự án: `pytest tests/test_m7_cert_lifecycle.py tests/test_m8_revocation_crl.py tests/test_infra_manager.py -v`. Để demo OCSP bằng dòng lệnh: `curl "http://localhost:8888/ocsp?serial=12345"`.

6 Tích hợp vào chương trình chung

Cụm này không đứng riêng mà ghép vào toàn hệ thống qua các điểm nối sau:

- **Khởi động app:** `ui/app.py` gọi `get_infra().start_prod_servers()` lúc boot để CRL + OCSP server Prod sẵn sàng phục vụ; khi thoát gọi `stop_all()`.
- **Phụ thuộc Root CA:** `publish_crl` và `renew_cert` gọi `load_active_root_ca_with_key` (module CA của thành viên khác). Nếu chưa có Root CA, hai hàm raise lỗi rõ ràng để UI hiển thị.
- **Nhúng URL vào cert:** khi cấp/renew cert, URL CRL + OCSP (`prod_crl_url()`, `prod_ocsp_url()` từ `infra_manager`) được nhúng vào extension CRL Distribution Points / AIA — để khâu *chain validation* (phần việc Phạm Minh Quân) biết đi tra ở đâu.
- **Audit log:** mọi hành động revoke/renew/approve/reject/publish đều gọi `services.audit.write_audit` với Action tương ứng (`CERT_REVOKED`, `CERT_RENEWED`, `REVOKE_APPROVED`, `CRL_PUBLISHED`...) — ghép vào màn Audit chung.
- **Chế độ remote:** các view Customer (`my_certs`, `revoke_request`, `view_crl`) hỗ trợ

gọi Admin API (`remote_csr_client`) khi chạy tách máy Admin/Customer — cùng một logic nghiệp vụ nhưng qua HTTP.

- **Cấu hình:** đường dẫn `PROD_CRL_PATH`, `PROD_OCSP_DB_PATH` và port lấy từ `config.py` (cho phép override qua biến môi trường để tránh xung đột port).

Quan hệ với chain validation. Đầu ra của cụm này (CRL đã ký phục vụ tại `/crl.pem` và OCSP responder tại `/ocsp`) chính là *đầu vào* cho bước kiểm tra trạng thái thu hồi trong quy trình verify của Phạm Minh Quân. Hai phần ghép với nhau tạo thành vòng tin cậy đầy đủ: cấp → thu hồi → công bố → kiểm tra.

7 Bổ sung sau rà soát: các hàm/feature dễ bị hỏi

- `unrevoke_serial(serial, ocsp_db_path)` — **gỡ thu hồi**. Đối xứng với `revoke_serial_ocsp_only`: loại serial khỏi OCSP DB (rollback, vd xóa server demo). Điểm nhấn: *CRL KHÔNG tự đổi* — phải *Publish CRL Now* mới đồng bộ. Đây là chiều ngược của lịch pha CRL vs OCSP (OCSP cập nhật tức thì, CRL theo chu kỳ).
- **Hai hàm liệt kê thu hồi song song.** `list_pending_revocations` (hardcode `WHERE status='pending'`) là tiện ích queue thuần; nhưng UI `revoke_queue_view` thực tế gọi `list_all_revocations(status=...)` (mặc định pending) để combobox cho xem cả approved/rejected/all.
- **Các API truy vấn trạng thái của InfraManager.** Ngoài `set_lab_ocsp_enabled: status()` trả dict 4 cờ để dashboard vẽ badge; `is_prod_running/is_lab_running` chỉ True khi *cả* CRL+OCSP chạy; `get_lab_ocsp_state` trả dict *mutable* để Lab bind `BooleanVar` toggle trực tiếp; `set_prod_ocsp_enabled` toggle Prod (chủ yếu cho test).
- **BOLA guard tầng 2** — `get_cert_detail(cert_id, owner_id=...)`. Khi customer xem chi tiết, truyền `owner_id` ⇒ thêm `AND c.owner_id = ?` và trả None nếu cert không thuộc về họ — chặn customer đoán `cert_id` để xem/tải cert người khác (ngoài BOLA ở `submit_revoke_request`).
- **Cặp ghi-đọc serial trong ocsp_db.json.** `save_revoked_list` cố ý ghi serial dạng *chuỗi* (`str(s)` — JSON không an toàn cho số nguyên rất lớn); `load_revoked_list` ép `int(s)` khi đọc. Cặp đối xứng này là lý do so sánh `serial in revoked` (set int) luôn đúng kiểu — khép kín cảnh báo “sai kiểu str/int luôn trả GOOD nhầm”.

8 Checklist chuẩn bị vấn đáp

- ☐ Giải thích được vì sao status cert tính động, đọc trôi chảy `_compute_status` và thứ tự ưu tiên `revoked/expired/active`.

- ☐ Phân biệt rõ CRL vs OCSP: cơ chế, ưu/nhược, cách bù nhau; chỉ ra trong code đâu là realtime (`sync_ocsp_db`) đâu là trễ (`publish_crl`).
- ☐ Nói được vì sao CRL phải ký bằng Root CA và OCSP DB thì không cần ký.
- ☐ Vẽ lại luồng thu hồi hai phía và chỉ ra điểm atomic + BOLA guard trong code.
- ☐ Phân biệt `renew_cert` (giữ subject + public key) với reissue qua CSR mới; giải thích `renewed_from_id`.
- ☐ Trình bày vòng đời 4 server, tính idempotent của start, và cơ chế toggle OCSP qua state dict mutable.
- ☐ Chạy được demo: revoke → publish → tra CRL/OCSP → tắt OCSP giả lập down (HTTP 503).
- ☐ Chạy được bộ test `test_m7_cert_lifecycle`, `test_m8_revocation_crl`, `test_infra_manager` và giải thích từng assert.
- ☐ Giải thích điểm nối với chain validation và lý do nhúng URL CRL/OCSP vào cert lúc phát hành.
- ☐ Trả lời được vì sao `serial_hex` cần `int(..., 16)` và rủi ro sai kiểu str/int trong so sánh OCSP.